

Where Does an LLM Fuzzer’s Domain Knowledge Come From?

Human, Code-Read, and Self-Acquired Knowledge for Differential Testing of Typed Deserializers

Ziyad Mohammad Mansy Ibrahim
Independent Researcher
ziyadmohammad37@gmail.com
ORCID: 0009-0008-3499-3828

Abstract—An LLM-guided refinement loop for differential testing of Dart JSON deserializers found divergence in 2.9% of generated documents on its own, but in 28.8% when a human gave it four observations from a manual characterization: the bottleneck was domain knowledge, and it came from a person. We ask whether the fuzzer can acquire that knowledge itself. Holding the loop fixed and varying only a 1,500-character knowledge slot, we compare four sources in a pre-registered study: none, the human’s observations, a summary written by an LLM agent that reads the deserializers’ code, and a summary written by an agent that designs and runs 20 black-box probe documents. On Dart ($n=5$ seeds per arm), human knowledge reaches 30.9%, code-read knowledge 11.6%, and no knowledge 1.5% (each pair $p \approx 0.008$, Holm-adjusted $p \approx 0.048$). Probing reaches 10.5% on average, but not detectably above no knowledge ($p \approx 0.69$): it is all-or-nothing, 49% on the one seed whose probes happened to hit a divergence and at most 1.3% on the other four. Which known divergence patterns each arm finds tracks exactly which facts its summary states. On a second ecosystem with no human characterization (Kotlin: Gson, Moshi, kotlinx.serialization, Jackson), where divergence is common, probing does help: 67.1% against 50.0% over ten seeds ($p \approx 0.007$, with a replication declared before it ran). Black-box self-characterization works when the behaviours worth knowing are easy to stumble on and fails when they are narrow, which is where knowledge matters most. Triage of the Kotlin divergences found no new bug in the libraries’ documented defaults, and one standards violation in kotlinx.serialization’s strict mode that had been reported only generically before.

Index Terms—fuzzing, differential testing, large language models, domain knowledge, black-box testing, JSON deserialization, Dart, Kotlin

I. Introduction

LLM-guided fuzzers are often described as bringing their own knowledge: a model trained on large amounts of code is expected to know what inputs stress a parser or a compiler [1], [2]. Our previous study of differential testing for Dart JSON deserialization [3] found the opposite in a controlled setting. A bounded refinement loop, in which an LLM writes and rewrites a Hypothesis [4] generator from a score-only feedback signal, found cross-implementation divergence in only 2.9% of generated documents, against

22.2% for a hand-built static generator. Richer feedback, incremental mutation and a larger model did not close the gap. Four observations taken from a thirteen-case manual characterization did: with them in the prompt, the same loop reached 28.8%. But it never found a divergence it had not been told about. The bottleneck was domain knowledge, and it came from a human.

This paper asks where else that knowledge can come from. A human characterization is slow and requires an expert; if a fuzzer is to be applied to a new ecosystem, the knowledge has to be obtained some other way. Two automated sources are natural. An agent can read the code of the implementations under test and summarize what it finds, as white-box LLM testing tools do [5], [6], [7]. Or an agent can run experiments: design a small number of probe documents, observe how each implementation handles them, and write down what it learned. The second is what the human did in our previous study, and it needs no source code, which matters when the implementations are closed, generated at compile time, or reflective. We call it black-box self-characterization.

We compare these sources under control. The refinement loop, model, sampling settings and feedback are those of our previous study, unchanged; the only thing that differs between arms is the text in a single knowledge slot of the prompt, capped at 1,500 characters. The four arms are none, human (the four observations, verbatim), code (a summary written after reading the four paths’ deserialization code) and probe (a summary written after running 20 probe documents in two rounds). The design, seeds, tests and decision rules were fixed and committed before any code or API spend, and every later change is recorded in a deviations log with its reason. We ask:

- RQ1 (acquisition). Can a probing agent acquire knowledge that closes the refinement gap as well as human knowledge does? We answer this on Dart, where the answer key is known.
- RQ2 (sources). How do the four knowledge sources compare in divergence rate, in which known diver-

gence patterns they lead to, and in cost?

- RQ3 (transfer and discovery). On an ecosystem with no human characterization, Kotlin with four widely used JSON libraries, does probing knowledge help, does it rediscover a known behaviour, and does it find anything new?

Our findings are:

- On Dart, knowledge sources are clearly ordered: human (30.9%) above code-read (11.6%) above none (1.5%), each difference significant after correction. Probing reaches 10.5% on average but is not distinguishable from no knowledge ($p \approx 0.69$). It is all-or-nothing: 49% on the one seed whose probes hit a divergence, at most 1.3% on the others.
- The divergence patterns each arm finds follow the facts its summary states. Every code summary states that two paths accept a non-integer id, and every code run finds the id pattern; no code summary states that one path substitutes an empty list for a missing tags field, and one code run in five finds that pattern. No probe summary mentions non-integer id values, and no probe run finds the id pattern.
- On Kotlin, where 11–14 of 20 probes diverge on every seed, probing knowledge helps: 67.1% against 50.0% over ten seeds ($p \approx 0.007$), using a replication declared before it ran.
- Triage of every Kotlin divergence class found documented defaults and known issues, including the Gson null-safety bypass that served as our answer key, and one standards violation in `kotlinx.serialization`’s strict mode, for which we prepared a focused report.

The overall lesson is conditional. Black-box self-characterization works when the behaviours worth knowing are easy to stumble on, and fails when they are narrow. Unfortunately, narrow behaviours are exactly where an unguided fuzzer also fails, so they are where knowledge matters most. The data, agents, prompts and analysis are available at <https://github.com/ziyadmansy/knowledge-sources-fuzzing>.

II. Related Work

LLM-driven fuzzing and test generation. LLMs have been used as zero-shot input generators for deep-learning libraries [1], as universal fuzzers across languages [2], and to escape coverage plateaus in search-based test generation [8]. These systems draw on what the model already knows from pre-training. Our earlier studies put such a model inside a bounded, coverage-free refinement loop, first against C parsers [9] and then against Dart deserializers with a differential oracle [3]; the second is where the knowledge gap studied here was measured.

Knowledge from reading code. WhiteFox [5] has an LLM agent read compiler optimization source code and write requirements for test programs that trigger it. Kernel-GPT [6] infers syscall specifications from kernel source for Syzkaller. DiffSpec [10] generates differential tests

from natural-language specifications and code artifacts. AIJon [7] generates IJON annotations from source and finds them roughly on par with human-written ones in a coverage-guided fuzzer. All of these derive knowledge by reading code or specifications, for coverage or for test requirements. None compares knowledge sources under control, and none acquires knowledge by running the target. Our code arm is a small, controlled instance of this family; our probe arm is the black-box alternative.

Differential testing of JSON processing. Differential testing [11] of JSON parsers across languages has found many semantic discrepancies at the parsing level, text to tree [12]. JsonATG [13] uses a multi-agent LLM pipeline to generate bug-triggering tests for Java JSON libraries. We test typed binding, tree to class, where divergences come from how each library maps JSON onto a language’s types and null-safety rules, and we compare four libraries on one class in two ecosystems.

Known Kotlin behaviour. That Gson bypasses Kotlin null-safety is a long-standing known issue [14], which we use as an answer key: a behaviour the agents should rediscover without being told.

III. Method

A. Targets and oracle

Both targets decode the same logical record: a non-null integer id, a string amount, a nullable string name, a status enum with three values, a non-null list of strings tags, and a nullable recursive child. Each generated document is run through four independent deserialization paths in one long-lived harness process. A document diverges if one path accepts it and another rejects it, or if all accept but decode different values. The primary metric, as in our previous study, is the percentage of all generated documents that diverge.

Dart reuses our previous harness unchanged [3]: hand-written parsing, `json_serializable` [15], `freezed` [16] and `built_value` [17], on Dart 3.13.2 [18]. All four receive the output of one shared `jsonDecode`, so invalid JSON is rejected identically before any path runs. Two divergence patterns are known (written in path order): RAAR, where the two paths that decode id through `num.toInt()` accept a large integer literal that `jsonDecode` has turned into a double, and saturate it; and RRRA, where `built_value` alone accepts a document with tags missing and substitutes an empty list.

Kotlin uses a new harness with the same protocol and four libraries at their documented defaults: Gson [19], Moshi with its generated adapter [20], `kotlinx.serialization` [21] and Jackson with its Kotlin module [22]. The record is an idiomatic data class with no default values and no configuration beyond the annotations each library needs to run. Unlike Dart, each library parses the raw text itself, so all four run on every input and syntax-level divergence counts. The known answer key is the Gson null-safety bypass [14]: Gson constructs

objects without Kotlin’s constructor checks, so a missing or null value for a non-null property is accepted as null. Divergence that follows from a documented default is labelled as such and never called a bug.

B. The refinement loop

The loop is our previous study’s score-only loop [3], unchanged except for one setting. Each of five iterations asks gpt-4.1-mini [23] (temperature 0.2) for a Hypothesis generator of JSON documents, draws 500 documents from it in a restricted sandbox, and reports back how many diverged. The prompt describes the schema and the four paths, gives the score and a generic strategy hint (vary one thing at a time: a wrong type, a missing field, a boundary value), and asks for a new generator. A test rebuilds all 100 prompts committed by the previous study’s score-only and knowledge arms byte for byte. The one change is the output-token limit, raised from 2,500 to 8,000 for every arm before any experiment data, after a pilot lost four of five generators to truncation; no experiment response was truncated.

C. Knowledge arms

Each arm fills one knowledge slot in the prompt: a one-sentence header naming the source, followed by a body of at most 1,500 characters.

None. The slot is empty.

Human (Dart only). The four observations from our previous study, verbatim (1,291 characters): that invalid JSON never reaches the paths; that `built_value` alone accepts a missing tags and uses an empty list; that wrong types and nulls are rejected by all four in every case tried; and that two paths decode `id` as `num.toInt()`, which saturates for integer literals outside the 64-bit range.

Code (Dart only). An agent reads a fixed, per-seed identical set of nine source files, about 17,000 characters: each path’s model and its generated *.g.dart code. It then writes a numbered list of behavioural observations. For Kotlin no comparable source exists: Gson and Jackson are reflective and `kotlinx.serialization` generates bytecode with a compiler plugin. That asymmetry is itself a practical argument for black-box probing.

Probe. An agent designs up to 20 probe documents in two rounds, at most 12 then the rest, written as raw text so that non-JSON probes are possible. Each probe runs through the real harness, and the agent sees, per path, whether it accepted or rejected the document, the exception type, and the decoded value. After the second round it writes a numbered list of observations.

Both agents use the same model and settings as the loop and see the same target description and strategy hint as the none arm, so each starts from exactly the none arm’s a priori information. If a summary is over the cap, one shortening call follows, then a cut at the last line break under the cap; the cut applied in 19 of 20 acquisitions (mean body 1,422 characters). Knowledge is

TABLE I
Dart divergence rate per run (% of all generated documents), seeds 700–704 in order.

Arm	Mean	Per seed
none	1.45	3.40, 0.00, 0.67, 3.20, 0.00
human	30.89	21.60, 29.65, 48.30, 29.13, 25.75
code	11.55	10.10, 16.04, 8.40, 13.40, 9.80
probe	10.51	1.07, 49.27, 1.27, 0.93, 0.00

acquired once per seed, so its quality varies across seeds, and that variance is part of the result.

D. Design and analysis

The study was pre-registered in a design document committed before any code or API spend; later changes are listed in a dated deviations log. Dart arms run on seeds 700–704 and Kotlin arms on 800–804. Pilots used seeds outside these ranges, and the probing and code agents’ prompts were frozen after one pilot change: both agents were given the refinement prompt’s existing strategy hint, which the none arm already sees.

For each run, iterations whose generator failed validation contribute no documents, as in our previous study. We compare arms with an exact two-sided Mann–Whitney U test and report the mean difference and Cliff’s δ . The primary test is probe against none on Dart, with probe against human co-primary; all six Dart pairs are reported with a Holm correction alongside raw p -values. The Kotlin primary is probe against none. A $p \geq 0.05$ is reported as “not detected at $n=5$ ”, not as “no effect”. A claim of comparability to human knowledge requires $|\delta| < 0.33$ in addition to $p \geq 0.05$. Replication on fresh seeds was allowed only if declared in the log before it ran. Secondary measures are pattern recall (the fraction of runs that find each known pattern at least once) and cost in calls and tokens.

IV. Results

A. RQ1 and RQ2: knowledge sources on Dart

Table I gives the divergence rate of every run. Human knowledge reproduces our previous result on fresh seeds (30.9%, against 28.8% before). Code-read knowledge sits between human and none, and every run in each of these three arms is ordered: all five human runs exceed all five code runs, which exceed all five none runs.

RQ1. Probe knowledge is not detectably better than none (Table II, $p \approx 0.69$, $\delta = +0.20$). By the pre-registered rule this is “not detected at $n=5$ ”, and the claim of comparability to human knowledge is not supported ($\delta = -0.60$). The mean hides the shape of the distribution. Four probe runs stay at or below 1.3%, in the range of none; one, seed 701, reaches 49.3%, above every human run. Seed 701 is the only seed whose probing phase observed a divergence at all: one of its 100 probes across five seeds diverged (`built_value` accepting a nested tags:

TABLE II

Pairwise Dart comparisons: difference in mean rate (percentage points), exact two-sided Mann–Whitney U, Holm-adjusted p , Cliff’s δ . $n=5$ per arm.

Comparison	Δ	p	p_{Holm}	δ
probe–none (primary)	+9.05	0.690	0.690	+0.20
probe–human (co-prim.)	−20.38	0.151	0.452	−0.60
human–none	+29.43	0.008	0.048	+1.00
code–none	+10.09	0.008	0.048	+1.00
code–human	−19.34	0.008	0.048	−1.00
probe–code	−1.04	0.151	0.452	−0.60

TABLE III

Dart pattern recall. Stated: summaries (of five) that state the fact behind the pattern. Found: runs (of five) that found the pattern at least once.

Arm	RAAR (id)		RRRA (tags)	
	stated	found	stated	found
none	—	3/5	—	3/5
human	5/5	5/5	5/5	5/5
code	5/5	5/5	0/5	1/5
probe	0/5	0/5	1/5*	4/5

*Seed 701 states that `built_value` accepts tags: null: related, but not the missing-field fact.

null), and its summary reported it. The probes of the other four seeds diverged on none of their 20 documents. When probing sees nothing, its summary is a confident description of agreement, and the loop behaves as if it had no knowledge. Summaries can also overgeneralize: one states that no implementation throws an undocumented exception, although three of the four paths throw Dart’s private `_TypeError` on the probes it ran.

RQ2. Human > code > none holds with the largest possible effect in every pair and survives the Holm correction. Code-read knowledge is the stable middle: it varies between 8.4% and 16.0% and never fails. Probe knowledge has a similar mean but the opposite profile. The two are not distinguishable at $n=5$.

What the knowledge contains decides what is found. Table III compares, for each Dart pattern, whether an arm’s knowledge states the fact behind it with how many runs found it. Every code summary states that two paths accept a non-integer id through `num.toInt()`; every code run finds RAAR. None of them mentions large integers or overflow, which is the other half of the mechanism, but a generator that varies id types reaches it anyway. No code summary states that `built_value` substitutes an empty list for a missing tags, and one code run finds RRRA. The probe summaries are the mirror image: none mentions a non-integer id, and no probe run finds RAAR, not even the one at 49%, whose divergences are all RRRA. No arm found a Dart pattern outside the answer key. The loop exploits what it is told; it does not extend it, which is what our previous study observed for human knowledge.

Cost. A none or human run takes five calls, about \$0.014 at list prices. Code knowledge adds two calls per

TABLE IV

Kotlin divergence rate (% of all documents), probe vs. none. The replication on seeds 805–809 was declared before it ran.

Seeds	none	probe	p	δ
800–804 (pre-registered)	45.77	64.34	0.095	+0.68
805–809 (replication)	54.14	69.92	0.056	+0.76
pooled, $n=10$ per arm	49.95	67.13	0.007	+0.70
pooled, RFC 8259-valid only	42.55	56.49	0.075	+0.48

seed (\$0.004) and probing four (\$0.006), plus 20 harness executions. Knowledge acquisition is cheap compared with the loop; the whole study, including pilots, cost less than one US dollar.

B. RQ3: Kotlin

On Kotlin the picture is reversed. The probing agents observed divergence on 11 to 14 of their 20 probes on every seed: the four libraries disagree about wrong types, nulls, unknown keys, unknown enum constants and lenient syntax. The summaries record these disagreements, and the loop uses them (Table IV).

The pre-registered comparison on seeds 800–804 had a large effect but did not reach significance ($p \approx 0.095$). As the design allowed, we declared a replication on fresh seeds before running it; it showed the same direction and size (+15.8 points, $\delta = +0.76$), and the pooled comparison over ten seeds per arm is significant ($p \approx 0.007$). We report all three rows: neither half is significant alone.

The last row restricts both arms to documents that are valid JSON under RFC 8259 [24]. The harness’s own validity label turned out to be too lenient during triage (Section IV-C), so this row uses a strict parser applied after the fact, as recorded in the deviations log. The effect shrinks ($\delta = +0.48$, $p \approx 0.075$): part of the probe arm’s advantage comes from malformed input that some libraries accept and others do not.

Every Kotlin run in both arms, including all ten without knowledge, rediscovered the Gson null-safety bypass, in 114 to 970 documents per run. On Kotlin this answer key is too easy to separate the arms.

C. Triage of Kotlin divergences

We grouped every divergent Kotlin document by pattern and exception types, reproduced each group by hand with a minimal document that changes one field of a valid record, and checked the result against each library’s documentation and issue tracker. No run produced a value divergence: whenever all four libraries accepted a document, they decoded the same value.

Eleven behaviours are documented defaults or known limitations. They are Gson’s null-safety bypass and its mapping of unknown enum constants to null; Gson and Jackson decoding a missing or null id as 0, which Jackson’s Kotlin module documents as a caveat; Gson, Moshi and Jackson accepting null elements in a `List<String>` (strict

null checks are opt-in for Jackson); the coercion of numbers and booleans to strings; `kotlinx.serialization` and Jackson rejecting unknown keys; `kotlinx.serialization` requiring a nullable property with no default; Jackson truncating 1.9 to 1; and Gson’s lenient syntax; and two coercions of string values to Long: Jackson reading an empty string as 0, and Gson and Moshi accepting a numeric string with a fraction, such as the string 1.0.

One behaviour is a standards violation. `kotlinx.serialization`’s default, non-lenient `Json` accepts raw control characters (U+0000–U+001F) inside strings, which RFC 8259 forbids and which Jackson and Dart’s `jsonDecode` reject. We reproduced it outside the harness on version 1.11.0 and on the latest release candidate, 1.12.0-RC. It had been reported only inside a generic benchmark issue that was closed as not planned without comment [25], and we have prepared a focused report. The same leniency is why the harness’s validity label, which used `kotlinx.serialization`’s parser, was not RFC-strict.

Two more defects surfaced only in our manual reproductions, not in any run or probe: Gson saturating an integer literal just above 2^{63} to `Long.MAX_VALUE` [26], and `kotlinx.serialization` wrapping integers modulo 2^{64} , so that 18446744073709551617 decodes as 1 [27]. Both are known and open. We do not credit them to the method: like RAAR for the probe arm, they sit in a narrow numeric region that neither the probes nor the generators visited.

V. Discussion

Probing is a search problem with a small budget. A probing agent learns only what its probes reveal. On Dart, where divergence requires a specific field to be absent or an integer literal to leave the 64-bit range, 20 probes chosen by the model’s prior found a divergence on one seed in five. Under the same generic hint, the probes varied one field at a time, but none of the 100 Dart probes removed a required top-level field or used a non-integer or out-of-range id. On Kotlin, where most one-field changes separate at least one library from the others, every seed found a dozen divergences. The agent’s ability to characterize a target is therefore bounded by the same thing that bounds the fuzzer: the density of interesting behaviour near the inputs the model finds natural. This is the conditional result in our title question. Self-acquired knowledge helps when an unguided search would have found some divergence anyway, and gives nothing where knowledge is most needed.

Negative evidence is misleading. A probe summary that observed no divergence does not say “I did not look at missing fields”; it says that the implementations behave identically, sometimes with an overgeneralization that is false. The loop then receives confident but empty knowledge. A practical probing agent should report what it did not test, or be required to cover categories of perturbation, such as each field missing, each field null and each numeric boundary, before summarizing. Such

a checklist is itself domain knowledge, so it moves the question rather than removing it, but it is generic across schemas and cheap to write once.

Code reading is reliable but incomplete. The code arm never failed, because the facts are in the code whether or not the agent thinks to test them. It stated the `num.toInt()` difference in every summary and found RAAR in every run. It missed `built_value`’s empty-list default, which lives in generated builder code rather than in the decoding call, and it never connected `toInt()` to `jsonDecode`’s treatment of large literals, which lives in the SDK and was not in its reading list. Reading more code would help, at the cost of the fixed budget. On Kotlin the arm could not be run at all: the equivalent logic is reflective or compiler-generated.

For practitioners. Where a human characterization exists, use it: it was the best source by a wide margin and costs nothing at run time. Where source is available, a code-reading summary gives a dependable improvement for a few cents. A probing agent is worth running first to learn whether disagreement is common; if its probes find none, treat its summary as uninformative, not as evidence of agreement.

A. Threats to validity

Sample size. Each arm has five seeds, the smallest size at which an exact Mann–Whitney test can reach $p < 0.05$. The primary Dart result is a non-detection, and the bimodal probe distribution means a larger sample would most likely estimate how often probing hits, not remove the variance. The Kotlin result relies on a pre-declared replication and is weaker on RFC-valid documents alone ($p \approx 0.075$).

Knowledge quality is uncontrolled. Summaries are capped at the same length but differ in accuracy, which is the variable under study. The cap was applied by cutting in 19 of 20 acquisitions, which drops the last observations; a different enforcement rule could change individual summaries.

One model family and one probe budget. All agents and the loop use `gpt-4.1-mini`, and the probe budget of 20 is a choice. A stronger model or a larger budget might probe more widely; our previous study found that a larger model alone did not close the gap.

Post-data change. The Kotlin harness’s validity label was found to be lenient after the data existed. It affects only a secondary metric, which we recompute with a strict parser; the harness is unchanged so that the committed runs stay reproducible. The output-token limit was raised before any experiment data, for every arm.

Construct. “Divergence” counts documents, and on Kotlin many of them express the same few documented behaviours. Pattern recall and the triage are reported alongside the rate for this reason.

VI. Conclusion

An LLM refinement loop for differential testing depends on domain knowledge it does not have. We compared

four sources of that knowledge under control, changing nothing but a 1,500-character slot in the prompt. A human characterization was best, a summary written from reading the code was a reliable second, and a summary written from 20 black-box probes was all-or-nothing on Dart, where divergence is rare, and clearly helpful on Kotlin, where it is common. Across arms, the divergence patterns found matched the facts the knowledge stated, and no arm went beyond them. Self-characterization is therefore not a substitute for knowledge where knowledge matters most; making it one will need probing that is systematic about what it covers and honest about what it did not.

Data Availability

The design and deviations log, harnesses, agents, prompts, every generated document and summary, and the analysis scripts are available at <https://github.com/ziyadmansy/knowledge-sources-fuzzing>.

References

- [1] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, “Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 423–435.
- [2] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, “Fuzz4All: Universal fuzzing with large language models,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024.
- [3] Z. M. M. Ibrahim, “Beyond sanitizers: LLM-guided refinement for differential JSON deserialization testing in Dart,” *Zenodo preprint*, 2026, artifact: <https://github.com/ziyadmansy/agent-ic-fuzzing-dart-json>.
- [4] D. R. MacIver, Z. Hatfield-Dodds, and Many other contributors, “Hypothesis: A new approach to property-based testing,” *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.
- [5] C. Yang, Y. Deng, R. Lu, J. Yao, J. Liu, R. Jabbarvand, and L. Zhang, “WhiteFox: White-box compiler fuzzing empowered by large language models,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA2, 2024.
- [6] C. Yang, Z. Zhao, and L. Zhang, “KernelGPT: Enhanced kernel fuzzing via large language models,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 2025, pp. 560–573.
- [7] J. M. Vadayath, H. Wang, M. Schloegel, J. Hu, W. Gibbs, T. Bao, A. Doupé, R. Wang, and Y. Shoshitaishvili, “AIJon: Automated generation of annotations for fuzzing,” *arXiv preprint arXiv:2609.18457*, 2026.
- [8] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, “CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models,” in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023.
- [9] Z. M. M. Ibrahim, “Coverage-free fuzzing: LLM-guided refinement of grammar-based test generators,” *Zenodo preprint*, 2026, artifact: <https://github.com/ziyadmansy/agent-ic-grammar-fuzzing>.
- [10] N. Rao, E. Gilbert, H. Green, T. Ramanananandro, N. Swamy, C. Le Goues, and S. Fakhoury, “DiffSpec: Differential testing with LLMs using natural language specifications and code artifacts,” *arXiv preprint arXiv:2410.04249*, 2024.
- [11] W. M. McKeeman, “Differential testing for software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.
- [12] J. Möller, F. Weißberg, L. Pirch, T. Eisenhofer, and K. Rieck, “Cross-language differential testing of JSON parsers,” in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, 2024.
- [13] S. Wang, Z. Zhong, S. Wen, and Y. Liu, “Multi-agent assisted automatic test generation for Java JSON libraries,” in *Proceedings of the 32nd Asia-Pacific Software Engineering Conference (APSEC)*, 2025, arXiv:2410.09414.
- [14] “GSON ignoring Kotlin data class null-safety and default value,” *Gson issue #1657*, <https://github.com/google/gson/issues/1657>, 2020.
- [15] “json_serializable: Automatically generate code for converting to and from JSON,” https://pub.dev/packages/json_serializable, 2026, version 6.14.1.
- [16] R. Rousselet, “freezed: Code generation for immutable classes,” <https://pub.dev/packages/freezed>, 2026, version 4.0.1.
- [17] “built_value: Value types with builders, Dart classes as enums, and serialization,” https://pub.dev/packages/built_value, 2026, version 8.13.0.
- [18] “The Dart programming language,” <https://dart.dev>, 2026, SDK version 3.13.2, accessed 2026.
- [19] “Gson: A Java serialization/deserialization library to convert Java objects into JSON and back,” <https://github.com/google/gson>, 2026, version 2.14.0.
- [20] “Moshi: A modern JSON library for Kotlin and Java,” <https://github.com/square/moshi>, 2026, version 1.15.2, with moshi-kotlin-codegen.
- [21] “kotlinx.serialization: Kotlin multiplatform / multi-format serialization,” <https://github.com/Kotlin/kotlinx.serialization>, 2026, version 1.11.0.
- [22] “jackson-module-kotlin: Module that adds support for serialization/deserialization of Kotlin classes,” <https://github.com/FasterXML/jackson-module-kotlin>, 2026, jackson 2.22.3.
- [23] OpenAI, “Introducing GPT-4.1 in the API,” <https://openai.com/index/gpt-4-1/>, 2025.
- [24] T. Bray, “The JavaScript Object Notation (JSON) data interchange format,” *RFC 8259*, IETF, 2017.
- [25] “[GLD serializer benchmark] Kotlin kotlinx-json 1.8.1 – spec compliance misses,” *kotlinx.serialization issue #3273*, <https://github.com/Kotlin/kotlinx.serialization/issues/3273>, 2026.
- [26] “Long upper limit value,” *Gson issue #1727*, <https://github.com/google/gson/issues/1727>, 2020.
- [27] “JsonPrimitive.longOrNull returns a wrapped value for integers above 2⁶⁴,” *kotlinx.serialization issue #3267*, <https://github.com/Kotlin/kotlinx.serialization/issues/3267>, 2026.